

Tema 2: Búsqueda en Espacio de Estados Planificación

José Luis Ruiz Reina
Miguel A. Gutiérrez Naranjo
Francisco J. Martín Mateos

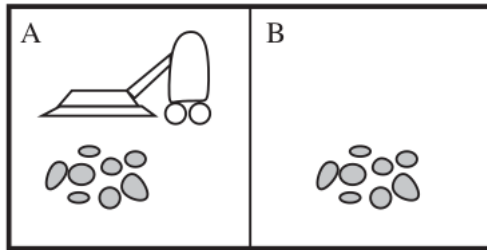
Departamento de Ciencias de la Computación e Inteligencia Artificial
Universidad de Sevilla

Inteligencia Artificial

Índice

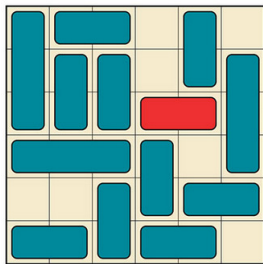
- Representación de problemas como espacios de estados
- Técnicas básicas de búsqueda de soluciones
- Búsqueda informada mediante técnicas heurísticas
- Introducción a la planificación
- El formalismo PDDL
- Búsqueda hacia adelante
- Búsqueda hacia atrás
- Heurísticas para planificación

Agentes “inteligentes”



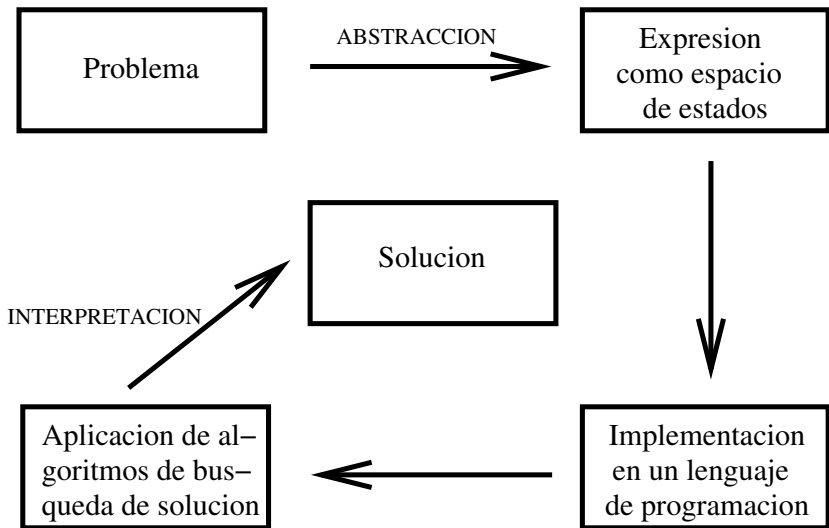
S. Russell y P. Norvig *Artificial Intelligence: A Modern Approach*

No es tan sencillo como parece



Necesita 51 movimientos

Método de solución de problemas



Definición de un problema como espacio de estados

- Paso previo a la búsqueda de una solución: **Especificación del problema**
 - describir de manera clara cada uno de los siguientes elementos
 - Ventaja: procedimientos generales de búsqueda de soluciones, *independientes* del problema
- Elementos del problema:
 1. ¿Cómo describir las diferentes situaciones o *estados* por los que podríamos pasar?
 2. ¿Cuál es el estado *inicial* desde la que se parte?
 3. ¿Cuál es el *objetivo final*? ¿qué debe verificar una situación para dar la búsqueda por finalizada?
 4. ¿Qué cambios elementales o *acciones* modifican los estados?

Problema: 8-puzzle

En un tablero cuadrado (3x3) hay situados 8 bloques numerados (dejando un hueco del tamaño de un bloque). Los bloques pueden deslizarse ocupando el hueco. El objetivo es ordenar los bloques.

2	8	3
1	6	4
7		5

Situación de partida
(varias opciones)

1	2	3
8		4
7	6	5

Objetivo

Espacio de estados: 8-puzle (estados) (I)

1. Estado: descripción de cualquier posible situación del problema

- Abstracción de propiedades
- Sólo considerar información relevante

La representación escogida influye en el número de estados y éste en la eficacia de los procedimientos de búsqueda de soluciones

8-puzle:

- relevante: localización de cada bloque y del hueco
- irrelevante: tipo de material de los bloques, colores de los bloques, . . .

Espacio de estados: 8-puzle (estados) (II)

Descripción de la posición exacta de cada uno de los bloques

2	8	3
7	1	4
	6	5

2		3
1	8	4
7	6	5

Representación vs. implementación

- Listas anidadas: `[[2, 8, 3], [7, 1, 4], ["H", 6, 5]]`
- Tuplas: `(2, 8, 3, 7, 1, 4, "H", 6, 5)`
- Diccionarios: `{primeraizquierda: 2, primeracentro: 8, ...}`

Número de estados: $9! = 362.880$

Espacio de estados: 8-puzzle (estado inicial)

2. Estado inicial: descripción de la situación de partida

8-puzzle:

2	8	3
1	6	4
7		5

Espacio de estados: 8-puzzle (objetivo)

3. Descripción del objetivo: Usualmente, un conjunto de estados, que llamaremos *finales* con alguna(s) característica(s) en común

8-puzzle: (un único estado final)

1	2	3
8		4
7	6	5

Espacio de estados: 8-puzle (acciones) (I)

4. Acciones: conjunto de opciones que permitirán transformar unos estados en otros
- Elementos que describen una acción
 - Aplicabilidad: precondition y postcondition
 - Efecto: transformación que una acción aplicable produce sobre un estado
 - Depende de la representación de los estados
 - Preferencia por representaciones con menor número de acciones

8-puzle:

- Según los movimientos de los bloques: $32 = (8 \text{ bl.} \times 4 \text{ mov.})$
- Según los movimientos del hueco: 4.

Espacio de estados: 8-puzzle (acciones) (II)

Representación

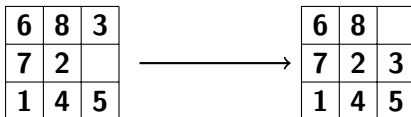
- Arriba
- Derecha
- Abajo
- Izquierda

Implementación

- $(0, -1)$
- $(1, 0)$
- $(0, 1)$
- $(-1, 0)$

Descripción de la primera: Arriba

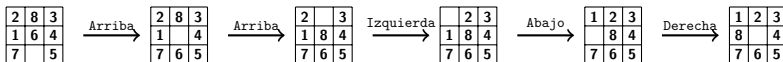
- Aplicabilidad: es aplicable a estados en los que el hueco no esté en la primera fila
- Efecto: intercambiar las posiciones del hueco y del bloque que está encima de éste



Solución de un problema

- Secuencia de acciones a realizar para conseguir el objetivo
- Secuencia de acciones cuya aplicación desde el estado inicial obtiene un estado final

8-puzzle: (no es única)



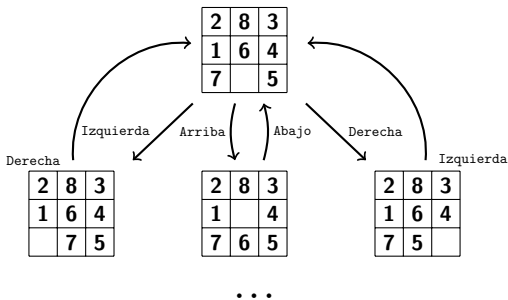
Tipos de planteamientos

- Proporcionar una solución.
- Determinar si existe un estado final alcanzable.
- Proporcionar cualquier solución lo más rápidamente posible.
- Proporcionar todas las soluciones.
- Proporcionar la solución más corta.
- Proporcionar la solución **menos costosa**.

Espacio de estados como grafo dirigido

- Los vértices del grafo son los distintos estados
- Los vecinos de un estado son aquellos que se pueden obtener a partir de una acción aplicable

8-puzzle



Problema: jarras (I)

Se tienen dos jarras, con capacidad para 4 y 3 litros respectivamente (sin marcas de medición) y un grifo; y necesitamos obtener 2 litros de agua.

1. Representación de estados: Contenido de las dos jarras

$\{4: j4, 3: j3\}$ con $j4$ en $\{0,1,2,3,4\}$ y $j3$ en $\{0,1,2,3\}$

Número de estados: 20.

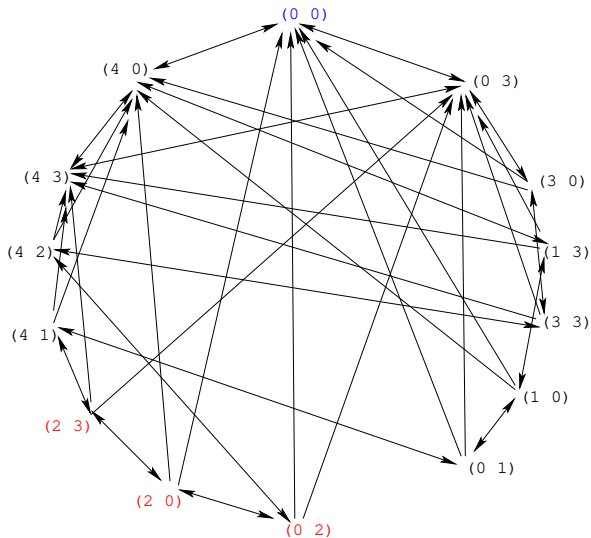
4. Acciones: Llenar o vaciar alguna de las jarras, trasvasar el contenido de una jarra en otra

(L 4) (L 3) (V 4) (V 3) (T 4) (T 3)

Problema: jarras (II)

2. Estado inicial: $\{4: 0, 3: 0\}$.
3. Estados finales: todos los estados $\{4: 2, 3: j3\}$ y $\{4: j4, 3: 2\}$.
4. Descripción de las acciones para $\{4: j4, 3: j3\}$:
 - (L i): $i = 4, 3$
 - Aplicabilidad: la jarra j_i no está llena ($j_i \neq i$).
 - Efecto: la jarra j_i está llena (si $i = 4$, pasaría a ser $\{4: 4, 3: j3\}$)
 - (V i): $i = 4, 3$
 - Aplicabilidad: la jarra j_i no está vacía ($j_i \neq 0$).
 - Efecto: la jarra j_i está vacía (si $i = 3$, pasaría a ser $\{4: j4, 3: 0\}$)
 - (T i): $i = 4, 3$
 - Aplicabilidad: la jarra j_i no está vacía y la otra no está llena.
 - Efecto: todo el contenido posible de la jarra j_i pasa a la otra.
(si $i = 4$ y hueco = $3 - j3$ pasaría a ser $\{4: j4 - \min(\text{hueco}, j4), 3: j3 + \min(\text{hueco}, j4)\}$)

Problema: jarras (III)



$(1, 1)$ $(1, 2)$ $(2, 1)$ $(2, 2)$ $(3, 1)$ $(3, 2)$

Problemas de la vida real

Problemas de la vida real que se pueden plantear y resolver como espacios de estados:

- Búsqueda de rutas en redes informáticas
- Diseño de rutas
- Problema del viajante
- Diseño de microchips
- Ensamblaje de componentes
- Desplazamiento de robots

Búsqueda de soluciones

Encontrar una secuencia de acciones que, partiendo del estado inicial, lleve a un estado final

Idea básica: exploración del *grafo* del espacio de estados

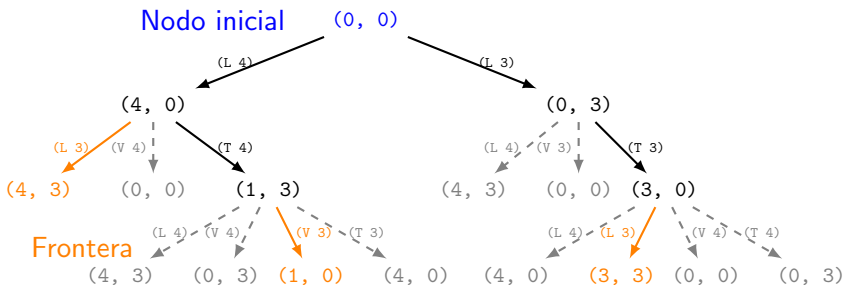
Repetir mientras haya estados que analizar: En cada momento se analiza un estado actual (en un principio, el inicial)

- Si el estado actual es final, acabar (devolviendo la secuencia de acciones de la rama a la que pertenece)
- En caso contrario, obtener los sucesores del estado actual (*expandir*)
- *Elegir* un nuevo estado actual, *dejando* los restantes para analizarlos posteriormente (si fuera necesario)

La elección del estado actual en cada momento determina una *estrategia de búsqueda*

Árboles de búsqueda

El proceso anterior puede verse como la construcción incremental de un árbol de búsqueda



Componentes de los nodos: **estado**, **padre** y **accion**

Elementos de la implementación

Elección de una representación (estructura de datos):

- para los estados
- para las acciones

Elementos que definen un problema

- **estado_inicial**: almacena la representación del estado inicial
- **es_estado_final** (ESTADO): comprueba si un estado es final
- **acciones_aplicables** (ESTADO): devuelve las acciones aplicables a un estado
- **aplicar** (ESTADO, ACCIÓN): obtiene el resultado de utilizar una acción aplicable sobre un estado

Pseudocódigo (procedimientos auxiliares)

`sucesor(nodo, acción)`

Devolver el nodo con

 estado : `aplicar`(estado(`nodo`), `acción`)

 padre : `nodo`

 acción : `acción`

`sucesores(nodo)`

Devolver el conjunto de nodos

 {sucesor(`nodo`, `acción`) |
 acción ∈ `acciones_aplicables`(estado(`nodo`)) }

Procedimiento general de búsqueda

búsqueda_genérica()

1. $\text{abiertos} \leftarrow \{(\text{estado} : \text{estado_inicial})\}$
 $\text{cerrados} \leftarrow \emptyset$
2. Mientras abiertos no esté vacía,
 - 2.1 $\text{actual} \leftarrow$ primer nodo de abiertos
 $\text{abiertos} \leftarrow$ resto de abiertos
 $\text{cerrados} \leftarrow \text{cerrados} \cup \{\text{estado}(\text{actual})\}$
 - 2.2 Si $\text{es_estado_final}(\text{estado}(\text{actual}))$,
 - 2.2.1 Devolver el camino desde la raíz a actual
 - 2.3 en caso contrario
 - 2.3.1 $\text{nuevos_sucesores} \leftarrow$ nodos de sucesores(actual)
 cuyo estado no está en ningún nodo de abiertos , ni en cerrados
3. Devolver Fallo.
 $\text{abiertos} \leftarrow \text{ORGANIZA}(\text{abiertos}, \text{nuevos_sucesores})$

Comentarios

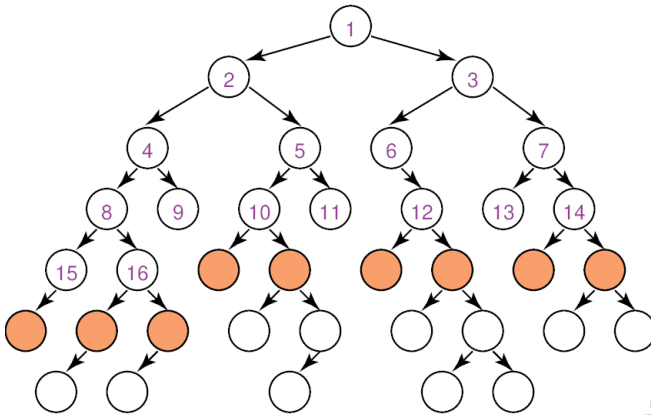
- La implementación anterior es *independiente del problema*
- **abiertos** puede verse como una “cola” en la que esperan los nodos para ser analizados
- **cerrados** contiene los estados de los nodos ya analizados:
 - Permite evitar ciclos en el proceso de búsqueda
 - Para determinados problemas es prescindible
- **ORGANIZA(abiertos, nuevos_sucesores)**:
 - Añade **nuevos_sucesores** a **abiertos**, reordenando según algún criterio concreto
 - Distintas concreciones de esta función dan lugar a distintos algoritmos de búsqueda (*estrategias* de búsqueda)

Propiedades de los algoritmos de búsqueda

- Adecuación: si devuelve una secuencia de acciones ¿es una solución del problema?
- Completitud: si existe solución, ¿encuentra alguna?
- Complejidad en tiempo: ¿cuánto se tarda en encontrar una solución? ¿cuántos nodos debe explorar?
- Complejidad en espacio: ¿cuánta memoria vamos a necesitar? ¿cuántos nodos alcanzan a tener entre **abiertos** y **cerrados**?

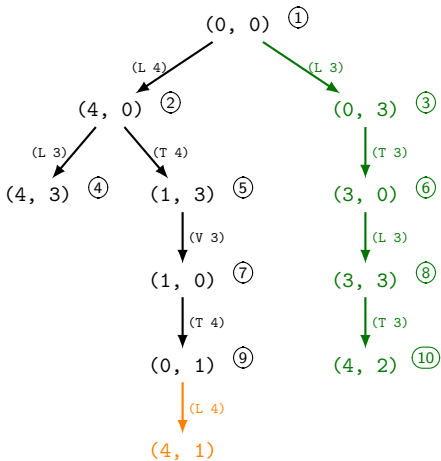
Búsqueda en anchura

En la búsqueda en anchura, **abiertos** se organiza como una cola (FIFO):



Búsqueda en anchura

Problema: jarras



Propiedades: anchura

- Es adecuada y completa
- Complejidad en tiempo y espacio $O(r^{p+1})$, donde:
 - r : factor de ramificación.
 - p : profundidad de la solución encontrada.

$$1 + r + r^2 + \dots + r^p = \frac{r^{p+1} - 1}{r - 1} = O(r^{p+1})$$

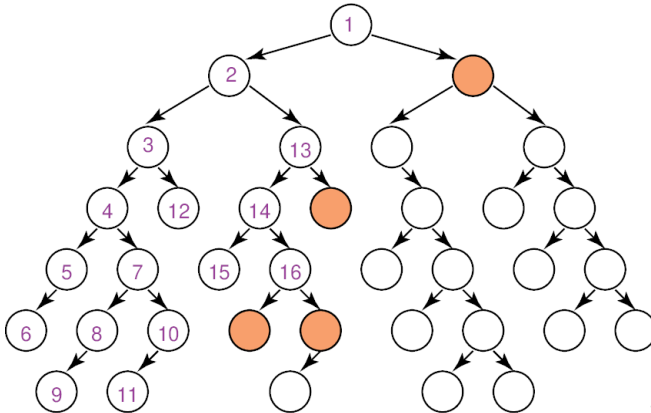
- Obtiene la solución con menor número de acciones.

Suponiendo ramificación **10**, **10^6** nodos por seg. y 1000 bytes por nodo:

Profundidad	Nodos~	Tiempo	Espacio
2	111	0.11 ms.	107 Kb
8	10^8	2 min.	103 Gb
16	10^{16}	350 años	10 Eb

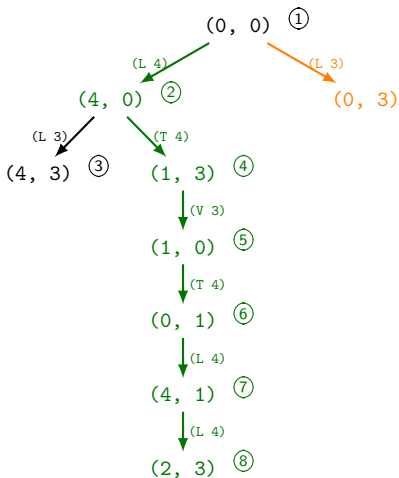
Búsqueda en profundidad

En la búsqueda en profundidad, **abiertos** se organiza como una pila (LIFO):



Búsqueda en profundidad

Problema: jarras



Propiedades: profundidad

- Es adecuada, pero no es completa
 - Sí lo es, si el espacio de estados es finito
- Complejidad en tiempo y espacio $O(r^{m+1})$, donde:
 - r : factor de ramificación.
 - m : máxima profundidad de la búsqueda (puede ser infinita).
- No necesariamente obtiene una solución mínima

Observaciones: complejidad espacial

La complejidad espacial, exponencial, la introduce la lista de **cerrados**

- En algunos espacios de estados no hay posibilidad de caminos cíclicos (y, por tanto, **cerrados** no es necesario)
- En general, para detectar ciclos sólo es necesario almacenar los estados de los nodos de la rama que se está explorando en cada momento

Es sencillo implementar una búsqueda en profundidad que, aún detectando ciclos, tenga complejidad espacial $O(r \cdot m)$

Observaciones: completitud (Iterativa)

Idea: no explorar caminos más allá de una determinada longitud.

Problema: si la cota establecida es menor que la longitud (desconocida en la mayoría de los casos) de la solución más corta

`búsqueda.en.profundidad.acotada(cota)`

1. `abiertos` $\leftarrow$ pila `{(estado : estado_inicial)}`
`cerrados` $\leftarrow \emptyset$
2. Mientras `abiertos` no esté vacía,
 - 2.1 `actual` $\leftarrow$ primer nodo de la pila `abiertos`
`abiertos` $\leftarrow$ resto de `abiertos`
`cerrados` $\leftarrow$ `cerrados` $\cup$ `{estado(actual)}`
 - 2.2 Si `es_estado_final(estado(actual))`,
 - 2.2.1 Devolver el camino desde la raíz a `actual`
 - 2.3 en caso contrario
 - 2.3.1 si `profundidad(actual)` es menor que `cota`
 - 2.3.1.1 `nuevos_sucesores` $\leftarrow$ nodos de `sucesores(actual)`
cuyo estado no está en ningún nodo de `abiertos`, ni en `cerrados`
`abiertos` $\leftarrow$ añadir `nuevos_sucesores` a la pila `abiertos`
3. Devolver Fallo.

Observaciones: completitud (Recursiva)

Idea: no explorar caminos más allá de una determinada longitud.

Problema: si la cota establecida es menor que la longitud (desconocida en la mayoría de los casos) de la solución más corta

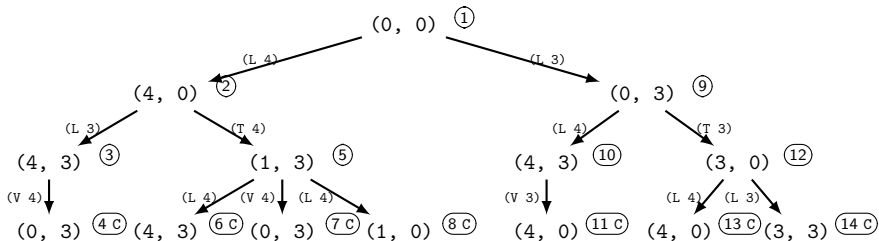
```
búsqueda.en.profundidad.acotada(cota)
Devolver bpa_recur_siva((estado : estado_inicial), cota, ∅)
```

```
bpa_recur_siva(nodo, cota, cerrados)
1. cerrados ← cerrados ∪ {estado(nodo)}
2. Si es_estado_final(estado(actual)),
   2.1 Devolver el camino desde la raíz a actual
3. en caso contrario, si cota = 0
   3.1 Devolver corte
4. en caso contrario
   4.1 hay_corte ← falso
      nuevos_sucesores ← nodos de sucesores(nodo)
      cuyo estado no está en cerrados
   4.2 Para cada nodo actual ∈ nuevos_sucesores
      4.2.1 resultado ← bpa_recur_siva(actual, cota-1, cerrados)
      4.2.2 Si resultado es corte
         4.2.2.1 hay_corte ← Verdadero
      4.2.3 en caso contrario, si resultado no es Fallo
         4.2.3.1 Devolver resultado
   4.3 Si hay_corte,
      4.3.1 Devolver corte
   4.4 en caso contrario
      4.4.1 Devolver Fallo
```

Búsqueda en profundidad acotada

Problema: jarras

cerrados sólo contiene los nodos de una rama.



Cota: 3

Propiedades: profundidad acotada

- Es adecuada, pero no es completa
- Complejidad en tiempo $O(r^c)$ y en espacio $O(r \cdot c)$, donde:
 - r : factor de ramificación
 - c : cota de la profundidad

Cuando no se conoce la cota, una opción para evitar la incompletitud es realizar búsquedas acotadas, incrementando la cota gradualmente

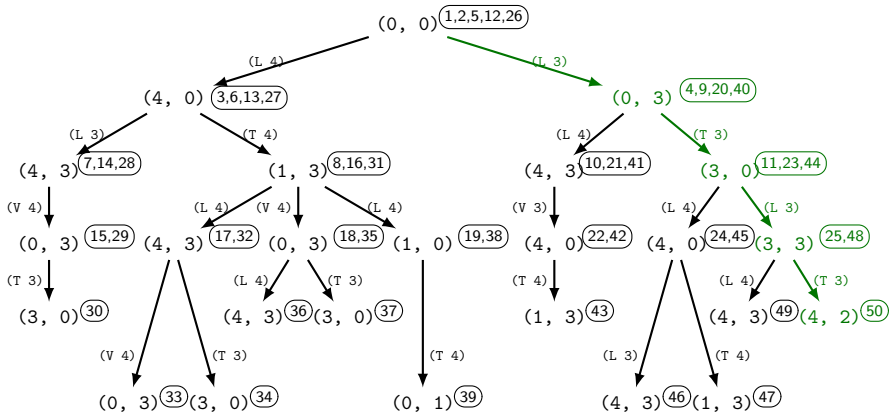
`búsqueda_en_profundidad_iterativa(estado_inicial)`

1. `cota` $\leftarrow$ 0
 `resultado` $\leftarrow$ corte
2. Mientras `resultado` sea corte
 - 2.1 `resultado` $\leftarrow$
 `búsqueda_en_profundidad_acotada(estado_inicial, cota)`
 `cota` $\leftarrow$ `cota`+1
3. Devolver `resultado`

Búsqueda en profundidad iterativa

Problema: jarras

cerrados sólo contiene los nodos de la rama.



Profundidad acotada, con cota **4**, exploraría **25** nodos

Propiedades: profundidad iterativa

- Es completa
- Complejidad en tiempo $O(r^p)$ y en espacio $O(r \cdot p)$, donde
 - r : factor de ramificación.
 - p : profundidad de la solución encontrada.
- Obtiene la solución con menor número de acciones.

Es el método preferido si el espacio de búsqueda es grande y no se conoce la profundidad de la solución

- La redundancia en la expansión queda compensada con la completitud. No es significativa: la mayoría de los nodos están en el último nivel del árbol

Ejemplo: $r = 10$ y $p = 5$:

- Búsqueda acotada, nodos analizados:
 $1 + 10 + 100 + 1000 + 10000 + 100000 = 111111$
- Búsqueda iterativa, nodos analizados (tan sólo un 10 % más):
 $1 + 11 + 111 + 1111 + 11111 + 111111 = 123456$

Comparación de procedimientos

	Anchura	Profundidad	Profundidad acotada	Profundidad iterativa
Tiempo	$O(r^{p+1})$	$O(r^{m+1})$	$O(r^{c+1})$	$O(r^{p+1})$
Espacio	$O(r^{p+1})$	$O(r \cdot m)$	$O(r \cdot c)$	$O(r \cdot p)$
Completa	Sí	No	Sí, si $p \leq c$	Sí
Mínima	Sí	No	No	Sí, si $c \leq pm$

- r : factor de ramificación.
- p : profundidad de la solución encontrada.
- pm : profundidad de la solución más corta.
- m : máxima profundidad de la búsqueda (puede ser infinita).
- c : cota de la profundidad.

Búsqueda informada

El conocimiento vendrá dado por una función que *estima* la “bondad” de los estados:

- Añadir un campo valor a los nodos

Idea: Dar preferencia a los estados mejores para educir el árbol de búsqueda y ganar *eficiencia*

Heurística: función numérica que valora los estados

- Siempre mayor o igual que 0 (en los estados finales: 0)

Todo el conocimiento específico que se va a usar sobre el problema está codificado en la función heurística.

```
sucesor(acción, nodo)
```

Devolver el nodo con

```
estado : aplicar(estado(nodo), acción)
padre  : nodo
acción : acción
valor  : heuristica(estado(nodo))
```

8-puzzle: heurísticas

1. **heurística (ESTADO)**: número de piezas descolocadas respecto de su posición en el estado final

2	8	3
1	6	4
7		5

valor:4

1	2	3
8		4
7	6	5

valor:0

2. **heurística (ESTADO)**: suma de las distancias de cada pieza a su posición en el estado final

2	8	3
1	6	4
7		5

valor:5

1	2	3
8		4
7	6	5

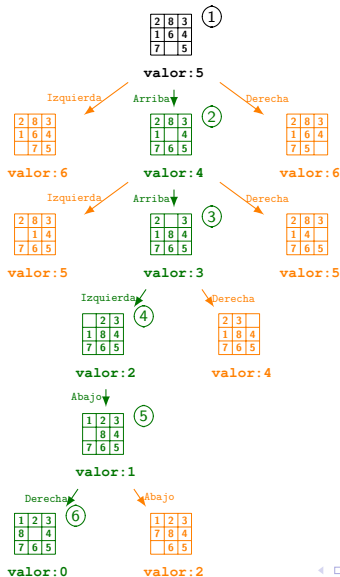
valor:0

Idea de la búsqueda por primero el mejor

- Búsqueda por primero el mejor:
 - Analizar preferentemente los nodos con heurística más baja.
 - Ordenar la cola de abiertos por heurística, de menor a mayor
- También llamada búsqueda *voraz* o *codiciosa* (del inglés “greedy”)
 - Porque siempre elige expandir *lo que estima* que está más “cerca” del objetivo
- Su rendimiento dependerá de la bondad de la heurística usada.

Búsqueda primero el mejor (II)

Problema: 8-puzzle (2^{a} heurística)



Propiedades: primero el mejor

- Complejidad en tiempo $O(r^m)$, donde:
 - r : factor de ramificación.
 - m : profundidad máxima del árbol de búsqueda.
- Complejidad en espacio: $O(r^m)$.
- En la práctica, el tiempo y espacio necesario depende del problema concreto y de la calidad de la heurística usada
- No es completa, en general.
 - Por ejemplo, una mala heurística podría hacer que se tomara un camino infinito.
- No es minimal (no garantiza soluciones con el menor número de acciones).
 - La heurística podría guiar hacia una solución no minimal

Invención de heurísticas

- Recordar: estimación de la “cercanía” a un estado final
- Técnicas para encontrar heurísticas
 - Relajación del problema
 - Combinación de heurísticas
 - Uso de información estadística
- Evaluación eficiente de la función heurística

Planificación en IA

- Planificar: encontrar una secuencia de acciones que alcanzan un determinado objetivo si se ejecutan desde un determinado estado inicial.
- Plan: secuencia de acciones que consiguen el objetivo
- Aplicaciones del mundo real:
 - Robótica
 - Fabricación mediante ensamblado de componentes
 - Misiones espaciales
- *Planning vs Scheduling*
 - *Planning* se refiere a la secuenciación de acciones para conseguir un objetivo y *Scheduling* pone más énfasis en la utilización eficiente de los recursos disponibles, por parte de las acciones

El problema de la planificación en IA

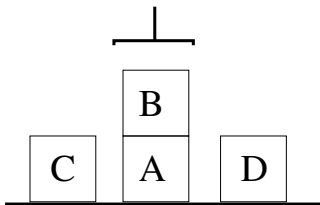
- Posibles cuestiones a abordar:
 - **Representación del mundo y de las acciones que lo transforman**
 - **Algoritmos de búsqueda de planes**
 - Minimizar los recursos consumidos por el plan
 - Tiempo en el que se realiza cada acción
 - Monitorizar la ejecución del plan, revisándolo en caso de errores o contingencias
- Por simplificar, en este tema supondremos:
 - Un número finito de estados
 - Completamente observables
 - Acciones deterministas, totalmente definidas por su especificación
 - Tiempo implícito: las acciones actúan *sin duración*
 - La planificación se realiza *a priori*
 - Hipótesis del mundo cerrado

Planificación y búsqueda en espacio de estados

- Problema abordable con búsqueda en espacio de estados, pero en problemas de escala real hay que abordar además:
 - Descripción de los estados en el mundo real extremadamente compleja
 - Gran cantidad de posibles acciones, muchas de ellas irrelevantes para la consecución del objetivo final
 - Las acciones sólo cambian una pequeña porción del mundo (*el problema del marco*)
 - Necesitamos heurísticas *independientes del dominio*
 - La necesidad de una acción puede detectarse sin necesidad de que se haya decidido las acciones previas (*compromiso mínimo*)
 - A veces es aconsejable *descomponer* en subproblemas más simples
- Idea: usar la lógica para representar estados, acciones y objetivos, y algoritmos que operan sobre esta representación

Ejemplo: el mundo de los bloques

- Ejemplo clásico en planificación.
- Elementos que intervienen:
 - Una superficie plana.
 - Una serie de bloques cúbicos.
 - Un brazo robotizado, que puede coger un bloque cada vez.
 - Un bloque puede estar sobre la mesa o apilado sobre otro bloque.



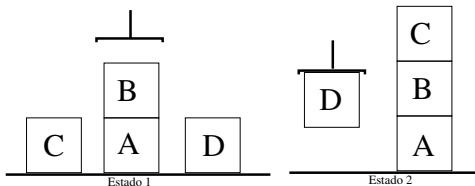
Formalismo lógico: el lenguaje PDDL

- Un lenguaje para representar problemas de planificación:
 - Constantes: objetos del mundo (en mayúsculas)
 - Variables para representar objetos genéricos (en minúsculas)
 - Símbolos de predicados (para expresar propiedades de los objetos)
 - Símbolos para representar las acciones
- Terminología:
 - Átomos: fórmulas de la forma $P(o_1, \dots, o_n)$, donde P es un símbolo de predicado y cada o_i es una constante o una variable (no hay símbolos de función)
 - Literales: átomos o negación de átomos (usaremos el símbolo $\neg$ para la negación)
 - Átomos y literales cerrados: sin variables
- Estados: conjunción de átomos cerrados
 - Hipótesis del mundo cerrado: los átomos que no se mencionan se suponen falsos

Representación de objetivos

- Objetivos: descripción de los estados finales.
- Los objetivos se representan como conjunción de literales (con posibilidad de usar variables)
 - Las variables en los objetivos se interpretan como existencialmente cuantificadas
- Satisfacer un objetivo:
 - Un estado satisface un objetivo si es posible sustituir las variables del objetivo por constantes de manera que el estado contiene todos los literales positivos del *objetivo instanciado* y ninguno de los negativos
 - Un estado es estado final si satisface el objetivo requerido
- Importante: verificar si un estado satisface un objetivo es un cálculo meramente simbólico

Ejemplo de objetivos en el mundo de los bloques



- Ejemplos de objetivos:
 - **SOBRE (B, A) , SOBRELAMESA (A) , -SOBRE (C, B)** es satisfecho por el estado 1 pero no por el estado 2
 - **SOBRE (x, A) , DESPEJADO (x) , BRAZOLIBRE ()** es satisfecho por el estado 1 pero no por el estado 2
 - **SOBRE (x, A) , SOBRE (y, x)** no es satisfecho por el estado 1 pero sí por el estado 2
 - El objetivo **SOBRE (x, A) , -SOBRE (C, x)** es satisfecho por el estado 1 pero no por el estado 2

Descripción de esquemas de acciones

- Para intentar solucionar el *problema del marco*, sólo se especifica lo que *cambia* por la acción.
- Una acción se describe mediante:
 - Su nombre y *todas* las variables involucradas: $O(x_1, \dots, x_n)$
 - Precondición: lista de literales que deben cumplirse para poder aplicar la acción.
 - Efectos: lista de literales que indica los cambios que se producirán cuando se aplique.

Descripción de esquemas de acciones

- En la lista de efectos distinguimos:
 - Efectos positivos (o lista de *adición*): átomos que pasarán a ser ciertos
 - Efectos negativos (o lista de *borrado*): átomos que dejarán de ser ciertos
- El uso de variables hace que una acción usualmente represente, en realidad, un *esquema de acción*:
 - Por cada manera de sustituir las variables por constantes, tenemos una *acción* concreta (una *instancia* de la acción)

Ejemplo: acciones en el mundo de los bloques

- Colocar un bloque sobre otro:

```
APILAR(x,y)
  Prec.: DESPEJADO(y), AGARRADO(x)
  Efec.: -DESPEJADO(y), -AGARRADO(x),
         BRAZOLIBRE(), SOBRE(x,y), DESPEJADO(x)
```

- Quitar un bloque que estaba sobre otro:

```
DESAPILAR(x,y)
  Prec.: SOBRE(x,y), DESPEJADO(x), BRAZOLIBRE()
  Efec.: -SOBRE(x,y), -DESPEJADO(x), -BRAZOLIBRE(),
         AGARRADO(x), DESPEJADO(y),
```

- Agarrar un bloque con el robot:

```
AGARRAR(x)
  Prec.: DESPEJADO(x), SOBRELAMESA(x), BRAZOLIBRE()
  Efec.: -DESPEJADO(x), -SOBRELAMESA(x), -BRAZOLIBRE(),
         AGARRADO(x)
```

- Bajar un bloque hasta la superficie:

```
BAJAR(x)
  Prec.: AGARRADO(x)
  Efec.: -AGARRADO(x),
         SOBRELAMESA(x), BRAZOLIBRE(), DESPEJADO(x)
```

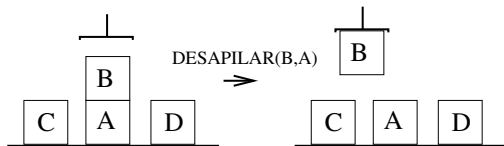
Aplicabilidad de una acción

- Una acción es *aplicable* a un estado si éste satisface su precondition
 - Si aparecen variables en la precondition, la aplicabilidad se define *respecto de la sustitución* θ usada para satisfacer la precondition
 - Por abreviar, a veces la sustitución usada aparecerá implícita al hablar de la acción
 - Por ejemplo, hablaremos de **DESAPILAR** (**A**, **B**) para referirnos a **DESAPILAR** (**x**, **y**) con la sustitución **[x/A, y/B]**
 - Un mismo esquema de acción puede dar lugar a distintas acciones que son aplicables al mismo estado

Resultado de aplicar una acción

- El resultado de aplicar una acción aplicable (respecto de una sustitución θ) a un estado E es el estado resultante de:
 - Eliminar de E los átomos, instanciados por θ , correspondiente a la lista de efectos negativos (si estuvieran)
 - Añadir a E los átomos, instanciados por θ , correspondientes a la lista de efectos positivos (si no estuvieran)

Ejemplo de aplicación de acción (I)



* Estado antes de aplicar DESAPILAR(B,A):

$E = \{ \text{DESPEJADO}(B), \text{DESPEJADO}(C), \text{DESPEJADO}(D), \text{BRAZOLIBRE}(), \\ \text{SOBRE}(B,A), \text{SOBRELAMESA}(C), \text{SOBRELAMESA}(D), \text{SOBRELAMESA}(A) \}$

* Precondiciones de DESAPILAR(B,A):

$\text{Prec} = \{ \text{SOBRE}(B,A), \text{DESPEJADO}(B), \text{BRAZOLIBRE}() \}$
----- Condiciones satisfechas en el estado -----
----- (acción aplicable) -----

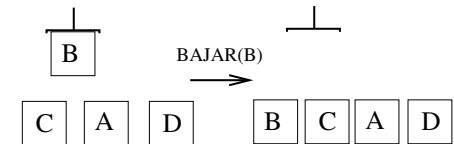
* Efectos de DESAPILAR(B,A):

$\text{Efec-} = \{ \text{SOBRE}(B,A), \text{DESPEJADO}(B), \text{BRAZOLIBRE}() \}$
 $\text{Efec+} = \{ \text{AGARRADO}(B), \text{DESPEJADO}(A) \}$

* Estado después de aplicar DESAPILAR(B,A):

$E' = (E \setminus \text{Efec-}) \cup \text{Efec+} = \\ \{ \text{DESPEJADO}(C), \text{DESPEJADO}(D), \text{SOBRELAMESA}(C), \\ \text{SOBRELAMESA}(D), \text{SOBRELAMESA}(A), \text{AGARRADO}(B), \text{DESPEJADO}(A) \}$

Ejemplo de aplicación de acción (II)



* Estado antes de aplicar BAJAR(B):

$E = \{ \text{DESPEJADO}(C), \text{DESPEJADO}(A), \text{DESPEJADO}(D), \text{SOBRELAMESA}(C), \text{SOBRELAMESA}(A), \text{SOBRELAMESA}(D), \text{AGARRADO}(B) \}$

* Precondiciones de BAJAR(B):

$\text{Prec} = \{ \text{AGARRADO}(B) \}$

----- Condiciones satisfechas en el estado -----
----- (acción aplicable) -----

* Efectos de BAJAR(B):

$\text{Efec-} = \{ \text{AGARRADO}(B) \}$

$\text{Efec+} = \{ \text{SOBRELAMESA}(B), \text{BRAZOLIBRE}(), \text{DESPEJADO}(B) \}$

* Estado después de aplicar BAJAR(B):

$E' = (E \setminus \text{Efec-}) \cup \text{Efec+} =$

$\{ \text{DESPEJADO}(C), \text{DESPEJADO}(A), \text{DESPEJADO}(D), \text{SOBRELAMESA}(C), \text{SOBRELAMESA}(A), \text{SOBRELAMESA}(D), \text{SOBRELAMESA}(B), \text{BRAZOLIBRE}(), \text{DESPEJADO}(B) \}$

Planes y soluciones

- Plan: secuencia de acciones
 - La primera aplicable al estado inicial y cada una de ellas aplicable al resultado de la anterior
- Solución: plan que a partir del estado inicial obtiene un estado que satisface el objetivo

Ejemplo: cambio de rueda pinchada

- Lenguaje:
 - Objetos: **RUEDA-REPUESTO, RUEDA-PINCHADA, EJE, MALETERO, SUELO**
 - Predicado: **EN (-, -)**
- Estado inicial:
EN (RUEDA-PINCHADA, EJE) , EN (RUEDA-REPUESTO, MALETERO)
- Estado final:
EN (RUEDA-REPUESTO, EJE)

Acciones en el cambio de rueda pinchada

- Sacar la rueda de repuesto del maletero:

QUITAR(RUEDA-REPUESTO,MALETERO)

Prec.: EN(RUEDA-REPUESTO,MALETERO)

Efec.: EN(RUEDA-REPUESTO,SUELO) , -EN(RUEDA-REPUESTO,MALETERO)

- Quitar la rueda pinchada del eje:

QUITAR(RUEDA-PINCHADA,EJE)

Prec.: EN(RUEDA-PINCHADA,EJE)

Efec.: EN(RUEDA-PINCHADA,SUELO) , -EN(RUEDA-PINCHADA,EJE)

- Colocar la rueda de repuesto en el eje:

PONER(RUEDA-REPUESTO,EJE)

Prec.: -EN(RUEDA-PINCHADA,EJE) , EN(RUEDA-REPUESTO,SUELO)

Efec.: -EN(RUEDA-REPUESTO,SUELO) , EN(RUEDA-REPUESTO,EJE)

- Dejar el coche solo hasta la mañana siguiente:

DEJARSOLO()

Prec.:

Efec.: -EN(RUEDA-REPUESTO,SUELO) , -EN(RUEDA-REPUESTO,EJE) ,
-EN(RUEDA-REPUESTO,MALETERO) -EN(RUEDA-PINCHADA,EJE) ,
-EN(RUEDA-PINCHADA,SUELO)

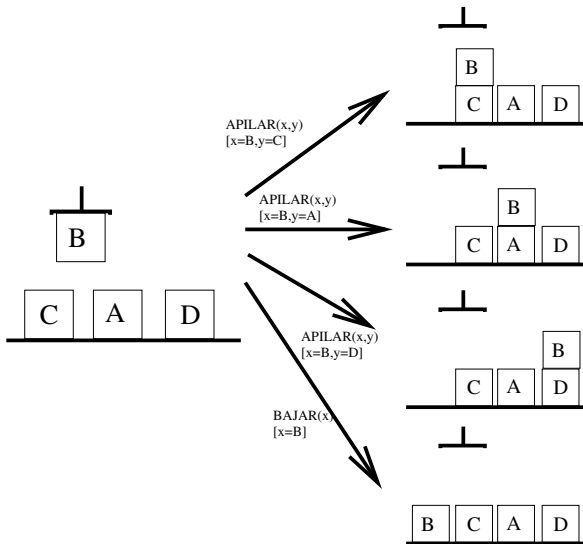
Extensiones al formalismo PDDL

- Existen sistemas de planificación que usan un lenguaje de representación más expresivo
 - Variables con tipos (no es una extensión que aumente la capacidad expresiva)
 - Uso de símbolos de función
 - Manejo del símbolo de igualdad
 - Evaluación de funciones al aplicar acciones
 - Disyunciones en las precondiciones
- En general existe un compromiso entre expresividad del lenguaje y simplicidad de los algoritmos que manejan la representación

Búsqueda en el espacio de estados

- Un problema de planificación se puede plantear como un problema de espacio de estados:
 - Estados descritos mediante listas de átomos cerrados.
 - Acciones como listas de precondiciones y efectos.
 - Función **es_estado_final** descrita por un objetivo.
- La búsqueda de planes podría hacerse usando los algoritmos de búsqueda ya vistos en los temas anteriores: anchura, profundidad, primero el mejor, ...
- El uso del formalismo lógico permite el uso de heurísticas *independientes del dominio*
 - Por ejemplo, el número de literales en el objetivo que quedan por satisfacer en un estado

Cálculo de sucesores



Búsqueda hacia adelante en profundidad

búsqueda_en_profundidad.h()

Devolver bep_h_rec($\emptyset$, $\emptyset$, estado_inicial)

bep_h_rec(plan, cerrados, estado)

1. Si estado satisface objetivo

1.1 Devolver plan

2. en caso contrario

2.1 aplicables $\leftarrow$ las instancias de las acciones aplicables
a estado cuya aplicación no sea un
elemento de cerrados

aplicables_ord $\leftarrow$ ordena_por_heuristica(aplicables)

2.2 Para cada accion $\in$ aplicables_ord

2.2.1 e $\leftarrow$ resultado de aplicar accion a estado

resultado $\leftarrow$ bep_h_rec(plan·accion, cerrados $\cup$ {e}, e)

2.2.2 Si resultado no es Fallo

2.2.2.1 Devolver resultado

2.3 Devolver Fallo

En el algoritmo anterior, queda por especificar la función

ordena_por_heuristica

- En general, se define a través de una heurística h que estima el número de pasos hasta el objetivo

Búsqueda hacia adelante en profundidad (propiedades)

- Es un algoritmo tipo *Backtracking* en el que se ordenan por heurística los sucesores del estado actual
- Propiedades:
 - Adecuado, completo y siempre termina. No se garantiza la solución más corta
 - Complejidad en tiempo: exponencial
 - La complejidad en espacio es lineal respecto de la máxima profundidad del árbol de búsqueda (esto lo hace preferible a primero el mejor)
- En la práctica, su eficiencia depende de la bondad de la heurística
 - Comentaremos más sobre heurísticas posteriormente

Búsqueda hacia atrás

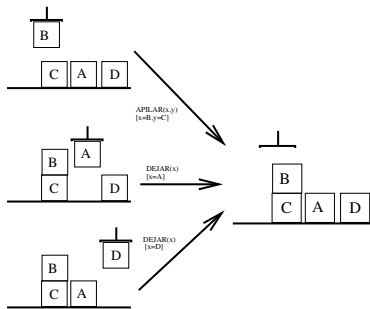
- Si la heurística no es buena, la búsqueda hacia adelante tiene el problema de la excesiva ramificación
 - Muchas acciones aplicables, pero la mayoría de ellas irrelevante para el objetivo
- Alternativa a la búsqueda hacia adelante: buscar hacia atrás, dirigidos por el objetivo
 - Se empieza en el objetivo, se aplican las acciones a la inversa y se trata de llegar al estado inicial
 - Nodos del árbol de búsqueda: objetivos
- La clave: sólo aplicar hacia atrás las acciones relevantes para el objetivo

Acción relevante para un objetivo

- Sea **G** un objetivo *sin variables* (más adelante trataremos el caso con variables). Decimos que una acción **A** es *relevante* para **G** si:
 - Al menos, uno de los efectos (positivo o negativo) de la acción está en **G** con el mismo "signo"
 - Ninguno de los efectos (positivo o negativo) de la acción aparece en **G** con distinto "signo"
- Intuitivamente: una acción es relevante si podría ser la última de un plan que llevara al objetivo

Acciones relevantes: ejemplo

- Objetivo: { **DESPEJADO (A)** , **DESPEJADO (B)** , **DESPEJADO (D)** , **SOBRE (B, C)** }
- Acciones relevantes: **APILAR (B, C)** , **DEJAR (A)** y **DEJAR (D)**



- Ejemplo de acción no relevante para el mismo objetivo:
APILAR (A, D) (aunque tiene a **DESPEJADO (A)** como efecto positivo sin embargo tiene como efecto negativo **-DESPEJADO (D)**)

Cálculo de predecesores

- Si **A** es una acción relevante para un objetivo **G** (sin variables), entonces el *objetivo predecesor* de **G** respecto a **A** es $(G \setminus \text{efectos}(A)) \cup \text{precond}(A)$

- Ejemplo:

$G = \{\text{DESPEJADO}(A), \text{DESPEJADO}(B), \text{DESPEJADO}(D), \text{SOBRE}(B,C)\}$

Predecesor respecto de la acción relevante **APILAR(B,C)**

$$\begin{aligned} G' &= (G \setminus \{-\text{DESPEJADO}(C), -\text{AGARRADO}(B), \text{BRAZOLIBRE}(), \\ &\quad \text{SOBRE}(B,C), \text{DESPEJADO}(B)\}) \\ &\quad \cup \text{DESPEJADO}(C), \text{AGARRADO}(B) \\ &= \{\text{DESPEJADO}(A), \text{DESPEJADO}(D), \text{DESPEJADO}(C), \text{AGARRADO}(B)\} \end{aligned}$$

- Nótese que el formalismo lógico nos permite calcular predecesores de manera sencilla

Búsqueda hacia atrás con heurística (sin variables)

búsqueda.hacia_atrás.h()

Devolver bha.h_rec($\emptyset$, $\emptyset$, objetivo)

bha.h_rec(plan, cerrados, obj_actual)

1. Si estado_inicial satisface obj_actual

1.1 Devolver plan

2. en caso contrario

2.1 relevantes $\leftarrow$ las instancias de las acciones relevantes
para obj_actual tales que el predecesor
no sea un elemento de cerrados

relevantes_ord $\leftarrow$ ordena_por_heuristica(relevantes)

2.2 Para cada accion $\in$ relevantes_ord

2.2.1 obj $\leftarrow$ predecesor de obj_actual respecto a accion
resultado $\leftarrow$ bha.h_rec(a.plan, cerrados $\cup$ {obj}, obj)

2.2.2 Si resultado no es Fallo

2.2.2.1 Devolver resultado

2.3 Devolver Fallo

En el algoritmo anterior, queda por especificar la función

ordena_por_heuristica

- En general, se define a través de una heurística h que estima el número de pasos desde el estado inicial al objetivo actual

Búsqueda hacia atrás con heurística (propiedades)

- Es un algoritmo de búsqueda en profundidad en un espacio de estados (el de los objetivos)
 - Incorpora heurística para ordenar objetivos
- Propiedades
 - Adecuado, completo y siempre termina. No se garantiza la solución más corta
 - Complejidad teórica: como en la búsqueda hacia adelante
- En la práctica, y nuevamente debido a la elevada ramificación, su eficiencia depende de la heurística usada

Heurísticas para planificación basadas en espacios de estados

- Una componente fundamental para la eficiencia práctica de los anteriores algoritmos es la heurística usada para ordenar los estados o los objetivos
- Esta heurística debe estimar distancia entre estados y objetivos (número de acciones necesarias)
 - En búsqueda hacia adelante: para cada estado, distancia hasta el objetivo
 - En búsqueda hacia atrás: para cada objetivo, distancia hasta el estado inicial
- Buscamos heurísticas independientes del dominio
 - Basadas en la representación lógica de estados, objetivos y acciones

Heurísticas basadas en relajar del problema

- Heurísticas obtenidas relajando algunas restricciones del problema y calculando el número de acciones necesarias en ese problema relajado
- Algunas ideas para relajar el problema:
 - Ignorar las precondiciones y/o efectos negativos de las acciones
 - Suponer que cada literal de un objetivo se alcanza de manera independiente: como si el número de acciones necesarias para alcanzar un objetivo fuera la *suma* del número de pasos necesario para alcanzar cada literal del objetivo
 - Suponer que el número de acciones necesarias para alcanzar un objetivo es el número de pasos *máximo* necesario para alcanzar *uno* de sus literales
 - Ignorar todas las precondiciones de las acciones
 - Ignorar determinados predicados

La heurística Δ_0

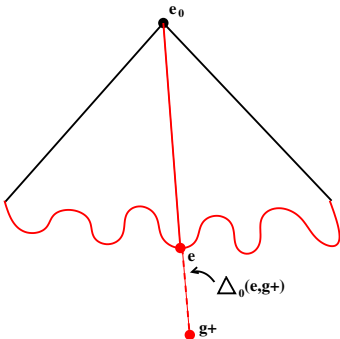
- Vamos a definir a continuación una heurística Δ_0 basada en las dos primeras ideas anteriores
- Dado un estado e , un átomo p y un objetivo g que sólo tiene literales positivos sin variables, definimos recursivamente $\Delta_0(e, p)$ y $\Delta_0(e, g)$ de la siguiente manera:
 - Si p aparece en e , $\Delta_0(e, p) = 0$
 - Si p no aparece en e ni en los efectos positivos de ninguna acción, $\Delta_0(e, p) = +\infty$
 - En otro caso, $\Delta_0(e, p) = \min_A \{1 + \sum_{q \in \text{precond}^+(A)} \Delta_0(e, q) \mid p \in \text{efectos}^+(A)\}$
 - $\Delta_0(e, g) = \sum_{p \in g} \Delta_0(e, p)$
- Notación: $\text{precond}^+(A)$ y $\text{efectos}^+(A)$ denotan respectivamente los literales positivos de la precondition y de los efectos de una acción A

La heurística Δ_0 (propiedades)

- Intuitivamente:
 - $\Delta_0(\mathbf{e}, \mathbf{p})$ cuenta el menor número de pasos necesarios para que se verifique $\mathbf{p}$ a partir de $\mathbf{e}$, suponiendo que las acciones no tienen precondiciones negativas ni efectos negativos
 - $\Delta_0(\mathbf{e}, \mathbf{g})$ es la suma de las anteriores estimaciones para cada átomo $\mathbf{p} \in \mathbf{g}$
 - No la usaremos con primero el mejor, sino con búsqueda en profundidad
- Dado un $\mathbf{e}$, los valores $\Delta_0(\mathbf{e}, \mathbf{p})$ para cada átomo $\mathbf{p}$ se pueden calcular con un algoritmo similar al algoritmo de caminos mínimos de Dijkstra

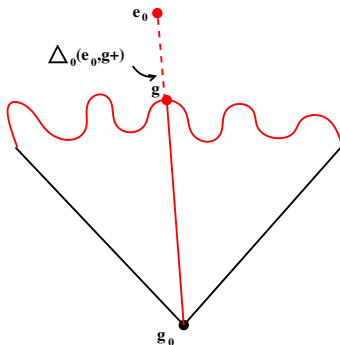
Usando la heurística Δ_0 en las búsquedas (I)

- En la búsqueda hacia adelante:
 - A cada acción A aplicable al estado actual se le asigna el valor heurístico $\Delta_0(e, g+)$, donde $g+$ es el conjunto de literales positivos del objetivo y e el estado que resulta al aplicar la acción A



Usando la heurística Δ_0 en las búsquedas (II)

- En la búsqueda hacia atrás:
 - A cada acción A relevante respecto del objetivo actual se le asigna el valor $\Delta_0(e_0, g+)$, donde e_0 es el estado inicial, g es el predecesor correspondiente a la acción A y $g+$ es el conjunto de literales positivos de g



Usando la heurística Δ_0 en las búsquedas (III)

- En el caso de que haya variables en los objetivos:
 - Si p es un átomo con variables, $\Delta_0(e, p)$ se define como el mínimo de entre todos los $\Delta_0(e, \sigma(p))$, siendo $\sigma(p)$ instancia sin variables de p
 - Si g es un objetivo con variables, $\Delta_0(e, g)$ es la suma de $\Delta_0(e, p)$ para cada literal positivo p de g

Otras aproximaciones a la planificación

- Planificación de orden parcial (POP)
- SATPLAN, uso de técnicas basadas en lógica proposicional
- GRAPHPLAN
- Planificación como PSR
- Planificación con recursos limitados
- Planificación con tiempo explícito
- Planificación mediante descomposición jerárquica
- Planificación en entornos con incertidumbre
- Ejecución del plan: vigilancia y replanificación
- Planificación continua
- Planificación multiagente

Bibliografía

- Russell, S. y Norvig, P. *Artificial Intelligence (A Modern Approach)* (Prentice–Hall, 2010). Third Edition
 - Cap. 10: “Planning”.
- Russell, S. y Norvig, P. *Inteligencia Artificial (un enfoque moderno)* (Pearson Educación, 2004). Segunda edición
 - Cap. 11: “Planificación”.
- Ghallab M., Nau D. y Traverso P. *Automated Planning: theory and practice* (Elsevier, 2004)
 - Sec. 2.3 “Classical representation”.
 - Cap. 4 “State space planning”.
 - Cap. 9 “Heuristics in planning”.